



LABORATORIO DI RETI INFORMATICHE

CORSO DI LAUREA TRIENNALE IN INGEGNERIA INFORMATICA
A.A. 2025/2026

Francesca Righetti
francesca.righetti@unipi.it



ESERCITAZIONE 7

Programmazione con i socket - II

Programma

- Server concorrente
- Socket bloccanti e non bloccanti
- I/O multiplexing
- Socket UDP

SERVER CONCORRENTE

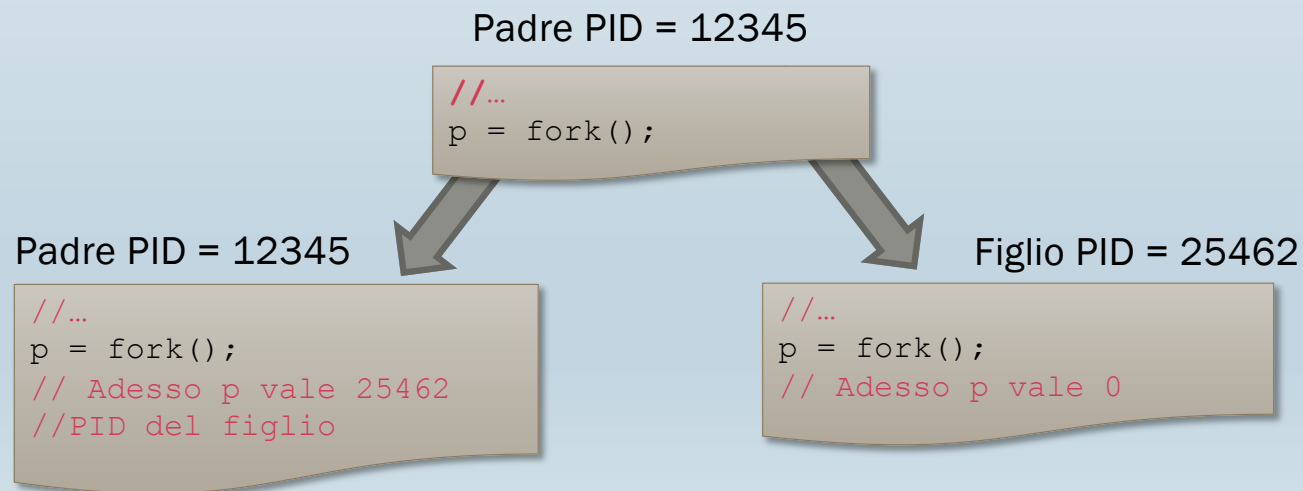
Tipi di server – Come può un server gestire più client contemporaneamente ?

- Server iterativo
 - Viene servita una richiesta alla volta
 - Il server accoda le richieste che arrivano nel frattempo
- Server concorrente
 - Serve più richieste «contemporaneamente»
 - Per ogni richiesta accettata (**accept()**) il server crea un nuovo processo figlio (o thread)

Server concorrente – Creazione di un processo figlio

```
#include <unistd.h>
pid_t fork(void);
```

- La primitiva **fork()** duplica il processo
 - Nel processo padre restituisce il PID del figlio
 - Nel processo figlio restituisce 0



Server concorrente - Uso di fork()

- Quando il processo viene duplicato, il padre e il figlio si ritrovano gli stessi descrittori, duplicati
- **Ognuno deve chiudere il descrittore che non usa, per evitare di mantenere aperte risorse non necessarie**
 - Il padre chiude il descrittore del socket connesso al client
 - Il figlio chiude il descrittore del socket in ascolto

Server concorrente – Server multi-processo

```
#include ...
int main () {
    int ret, sd, new_sd, len;
    struct sockaddr_in my_addr, cl_addr;
    //...
    pid_t pid;
    sd = socket(AF_INET, SOCK_STREAM, 0);
    ret = bind(sd, (struct sockaddr*)&my_addr, sizeof(my_addr));
    ret = listen(sd, 10);
    len = sizeof(cl_addr);
    //...
    while (1) {
        new_sd = accept(sd, (struct sockaddr*)&cl_addr, &len);
        pid = fork();
        if (pid == -1) { /*Gestione errore*/ }
        if (pid == 0) {
            // Sono nel processo figlio. Chiudo socket in ascolto
            close(sd);
            // Servo la richiesta con new_sd
            //...
            close(new_sd);
            exit(0); // Il figlio termina
        }
        // Sono nel processo padre. Chiudo socket connesso al client
        close(new_sd);
    }
}
```


Server concorrente - Uso dei thread

- Libreria da includere:
 - `#include <pthread.h>`
- Comando di compilazione (non sempre necessario)
 - `gcc <opzioni> file.c -pthread`
- **Tipi e funzioni da usare**
 - Un thread è identificato dal tipo *pthread_t*
 - Il semaforo per proteggere risorse condivise *pthread_mutex_t*
 - `int pthread_create(pthread_t* thread, const pthread_attr_t* attr, void* (*start_routine)(void *), void* arg )`
 - crea un nuovo thread che parte eseguendo `start_routine(void*)`
 - `int pthread_join(pthread_t thread, void** retval)`
 - un thread può bloccarsi in attesa della terminazione di un altro thread
 - `void pthread_exit(void* retval)`
 - per terminare un thread e mettere a disposizione un suo risultato (`retval`)

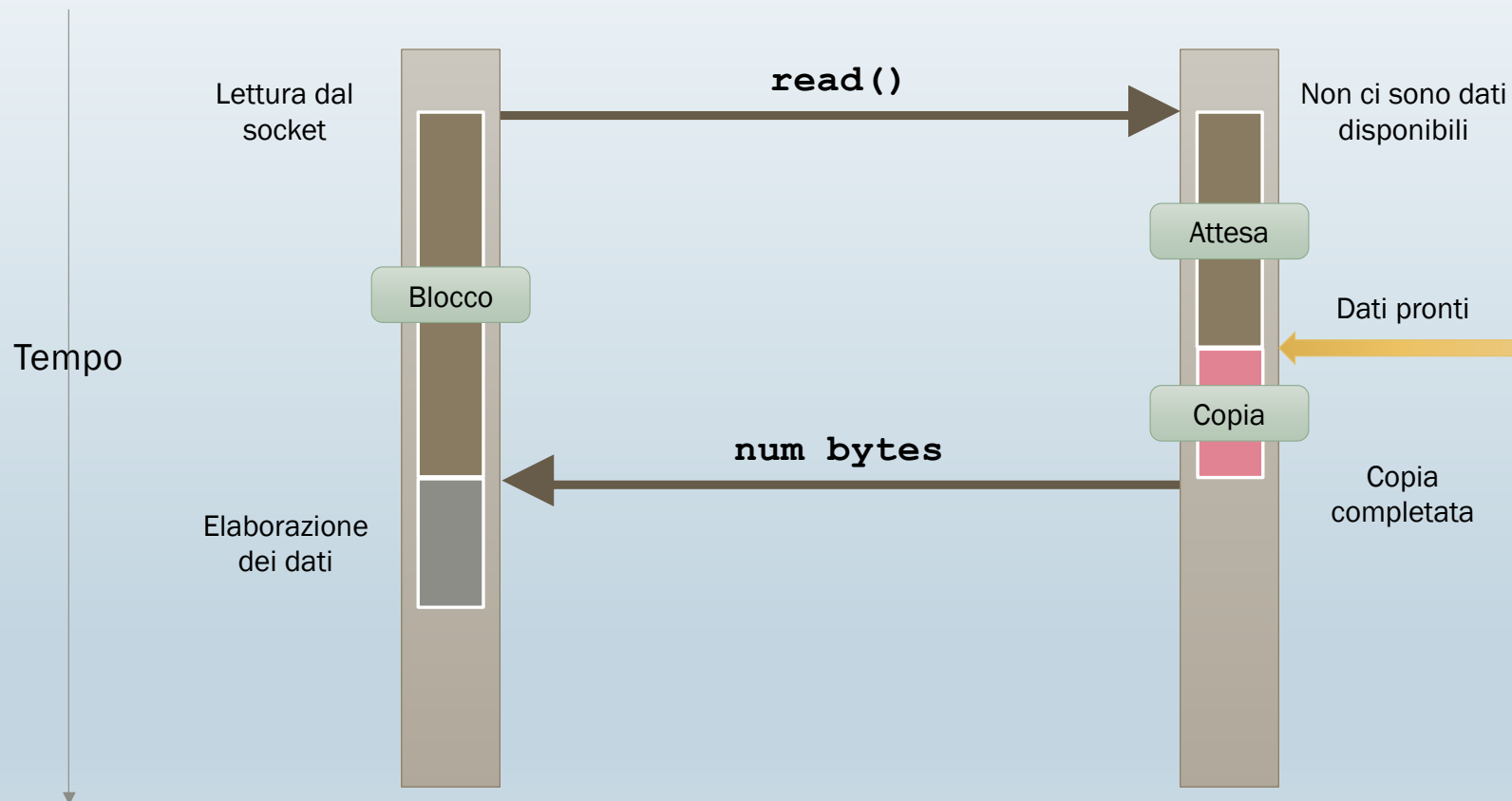
MODELLI DI I/O

SOCKET BLOCCANTI E NON BLOCCANTI

Socket bloccante

- Di default, un socket è **bloccante**
 - Tutte le operazioni su di esso fermano l'esecuzione del processo in attesa del risultato
 - **connect()** si blocca finché il socket non è connesso
 - **accept()** si blocca finché non c'è una richiesta di connessione
 - **send()** si blocca finché tutto il messaggio non è stato inviato (il buffer di invio potrebbe essere pieno)
 - **recv()** si blocca finché non c'è qualche dato disponibile
 - O finché tutto il messaggio richiesto non è disponibile, con il flag **MSG_WAITALL**

Socket bloccante



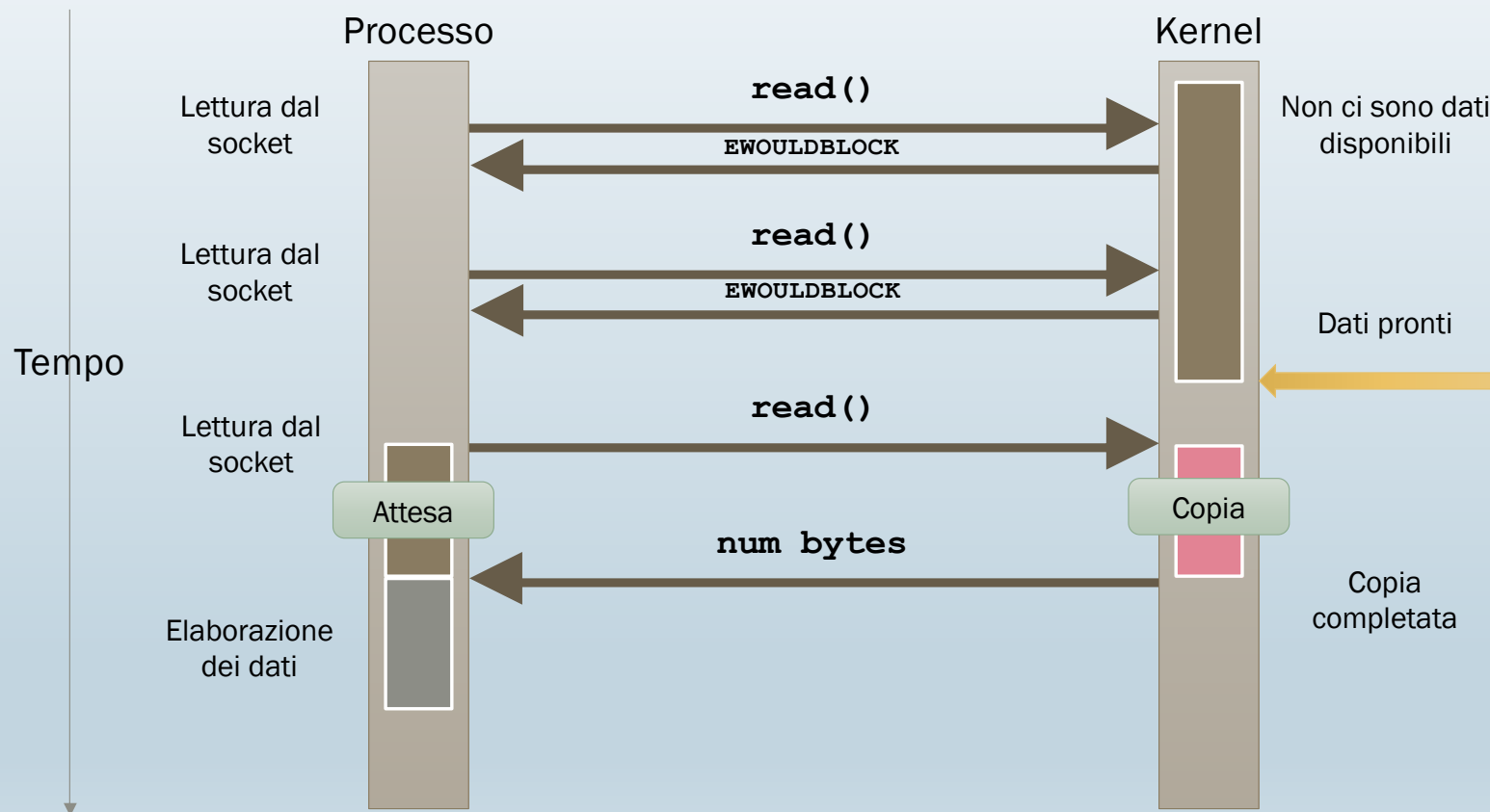
Socket non bloccante

- Un socket può essere settato come **non bloccante**
 - Le operazioni non attendono i risultati
 - **connect()** se non può connettersi subito restituisce -1 e setta **errno** a **EINPROGRESS**
 - **accept()** se non ci sono richieste restituisce -1 e setta **errno** a **EWOULDBLOCK**
 - **send()** se non riesce a inviare tutto il messaggio subito (il buffer è pieno), restituisce -1 e setta **errno** a **EWOULDBLOCK**
 - **recv()** se non ci sono messaggi restituisce -1 e setta **errno** a **EWOULDBLOCK**

```
socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);
```

Valore	Nome costante	Significato	Quando capita tipicamente
9	EBADF	Bad file descriptor	Il socket non è valido o è stato già chiuso.
13	EACCES	Permission denied	Porta privilegiata (<1024) senza privilegi root.
98	EADDRINUSE	Address already in use	bind() su una porta già occupata.
99	EADDRNOTAVAIL	Cannot assign requested address	IP non valido o non configurato sull'host.
111	ECONNREFUSED	Connection refused	Il server non è in ascolto sulla porta indicata.
113	EHOSTUNREACH	No route to host	Nessuna rotta verso l'host remoto.
110	ETIMEDOUT	Connection timed out	Nessuna risposta dal server entro il timeout.
32	EPIPE	Broken pipe	Scrittura (send) su un socket chiuso dall'altro lato.
104	ECONNRESET	Connection reset by peer	L'altro lato ha chiuso in modo "brusco" la connessione.
105	ENOBUFS	No buffer space available	Buffer di rete saturi, sistema sotto pressione.
115	EINPROGRESS	Operation now in progress	In socket non-bloccanti durante connect().
11	EAGAIN/EWOULDBLOCK	Resource temporarily unavailable	Nessun dato pronto in socket non-bloccante (o coda piena in send()).
4	EINTR	Interrupted system call	La recv()/send() è stata interrotta da un segnale.

Socket non bloccante



Socket bloccante Vs. non bloccante

- La differenza tra queste modalità riguarda il comportamento delle chiamate di I/O (come `read()`, `write()`, `accept()`, ecc.)
- La modalità bloccante è semplice da usare e va bene per applicazioni in cui i blocchi temporanei non sono problematici o in cui viene utilizzato il multithreading per gestire la concorrenza
- La modalità non bloccante è utile in applicazioni che devono gestire molte connessioni simultaneamente (come server web o server di chat) e dove bloccare il programma per un singolo socket sarebbe inefficiente
 - Viene utilizzata insieme a tecniche come il multiplexing di I/O, con chiamate come `select()`, `poll()`, o `epoll()`, per monitorare più socket e agire quando uno o più di essi diventano pronti per operazioni di I/O.

Socket bloccante Vs. non bloccante

Caratteristica	Socket Bloccante	Socket Non Bloccante
Lettura/Scrittura	Si blocca finché l'operazione non è completata	Ritorna immediatamente se l'operazione non può essere completata
Accept	Si blocca finché non arriva una connessione	Ritorna immediatamente se non c'è nessuna connessione disponibile
Gestione della concorrenza	Semplice, ma può richiedere multithreading/multiprocessing	Richiede gestione esplicita della concorrenza con <code>select()</code> , <code>poll()</code> , <code>epoll()</code>
Efficienza	Meno efficiente con molte connessioni	Più efficiente per applicazioni ad alta concorrenza

I/O MULTIPLEXING

I/O Multiplexing sincrono

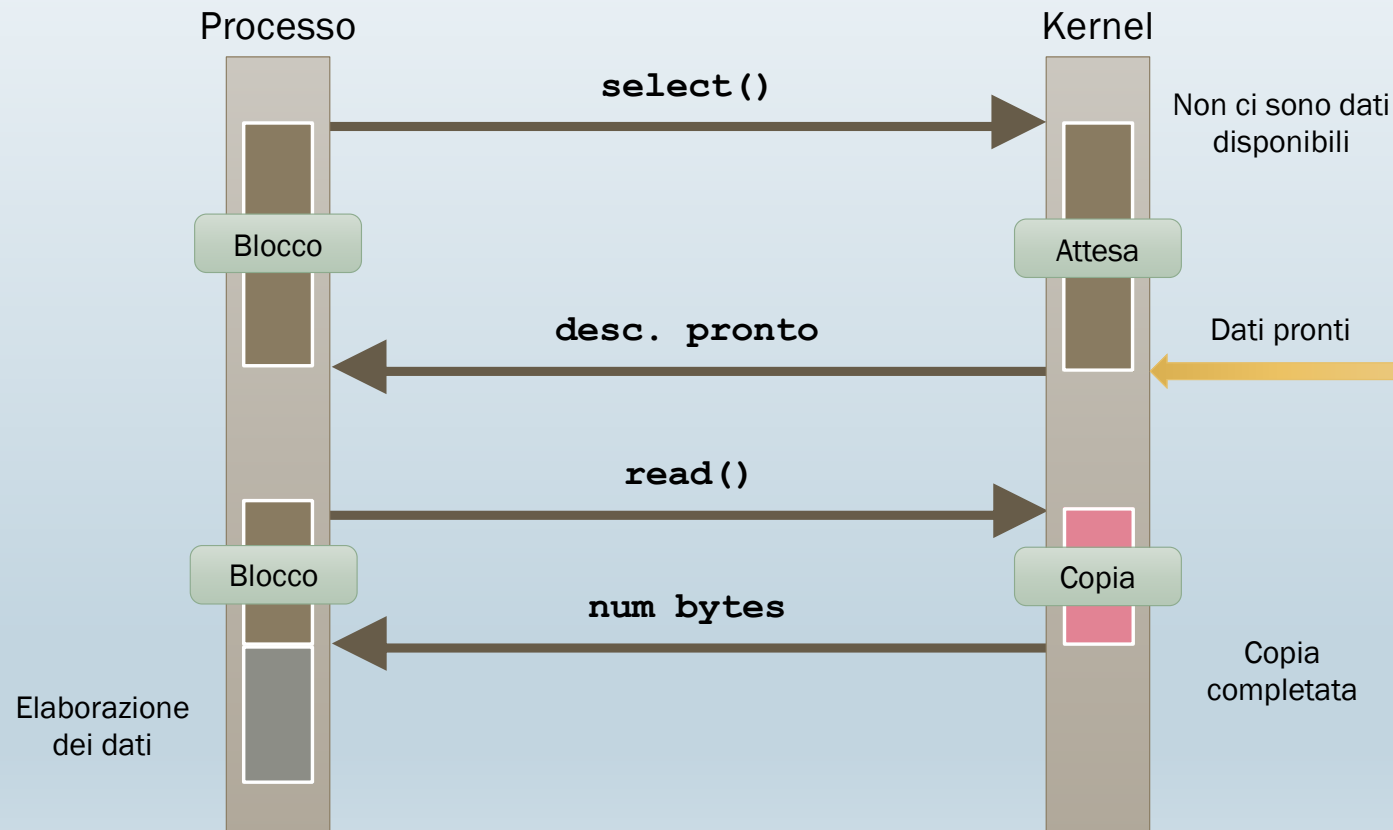
■ Problema:

- Voglio controllare più descrittori/socket nello stesso momento
- Se faccio operazioni su un socket bloccante, non posso controllarne altri

■ Soluzione:

- Multiplexing con la primitiva `select()`
- Esamina più socket contemporaneamente, il primo che è pronto viene usato
 - Sincrono -> Il programma rimane comunque in attesa che qualche descrittore sia pronto

I/O Multiplexing sincrono



I/O Multiplexing– Primitiva select()

- Controlla più socket contemporaneamente

```
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>

int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
```

- **nfds**: numero del descrittore più alto tra quelli da controllare, +1
- **readfds**: lista di descrittori da controllare per la lettura
- **writefds**: lista di descrittori da controllare per la scrittura
- **exceptfds**: lista di descrittori da controllare per le eccezioni (non ci interessa)
- **timeout**: intervallo di timeout
- La funzione restituisce il **numero di descrittori pronti**, -1 su errore
- **La funzione è bloccante**: si blocca finché un descrittore tra quelli controllati diventa pronti oppure finché il timeout non scade. Ma se scade il timeout ritorna 0 e non errore!

I/O Multiplexing– Descrittori pronti

- **select()** rileva i socket pronti
- Un socket è pronto in lettura se:
 - C'è almeno un byte da leggere
 - Il socket è stato chiuso (**read()** restituirà **0**)
 - È un socket in ascolto e ci sono connessioni effettuate
 - C'è un errore (**read()** restituirà **-1**)
- Un socket è pronto in scrittura se:
 - C'è spazio nel buffer per scrivere
 - C'è un errore (**write()** restituirà **-1**)
 - Se il socket è chiuso, **errno = EPIPE**

I/O Multiplexing– Struttura timeout

```
#include <sys/socket.h>
#include <netinet/in.h>

struct timeval {
    long tv_sec;      /* seconds */
    long tv_usec;     /* microseconds */
};
```

- **timeout = NULL**
 - Attesa indefinita, fino a quando un descrittore è pronto
- **timeout = { 10; 5; }**
 - Attesa massima di 10 secondi e 5 microsecondi
- **timeout = { 0; 0; }**
 - Attesa nulla, controlla i descrittori ed esce immediatamente (polling)

I/O Multiplexing– Insieme di descrittori

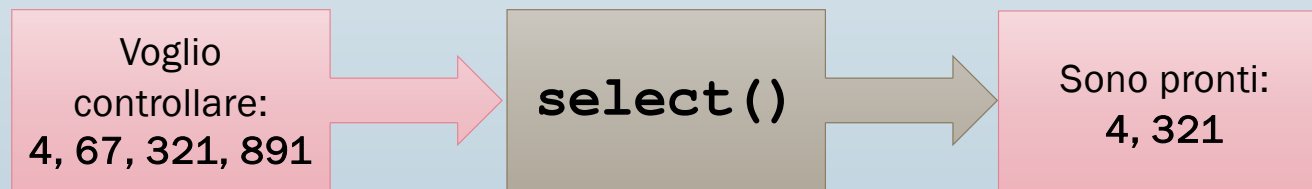
- Un descrittore è un int che va da 0 a FD_SETSIZE (di solito 1024)
- Un insieme di descrittori si rappresenta con una variabile di tipo fd_set
 - Non ci interessano i dettagli implementativi
 - Si manipola con delle macro simili a funzioni

```
/* Rimuovere un descrittore dal set */  
void FD_CLR(int fd, fd_set *set);  
/* Controllare se un descrittore è nel set */  
int  FD_ISSET(int fd, fd_set *set);  
/* Aggiungere un descrittore al set */  
void FD_SET(int fd, fd_set *set);  
/* Svuotare il set */  
void FD_ZERO(fd_set *set);
```

I/O Multiplexing– Insieme di descrittori

- **select()** modifica i set di descrittori:

- Prima di chiamare **select()** inserisco nei set di lettura e di scrittura i descrittori che voglio monitorare
- Dopo **select()** trovo nei set di lettura e scrittura i descrittori pronti



I/O Multiplexing– Utilizzo di select() (1/2)

```
int main(int argc, char *argv[]){
    fd_set master;           // Set principale
    fd_set read_fds;        // Set di lettura
    int fdmax;               // Numero max di descrittori

    struct sockaddr_in sv_addr; // Indirizzo server
    struct sockaddr_in cl_addr;  // Indirizzo client
    int listener;               // Socket per l'ascolto
    int newfd;
    char buf[1024];            // Buffer
    int nbytes;
    int addrlen;
    int i;

    /* Azzero i set */
    FD_ZERO(&master);
    FD_ZERO(&read_fds);

    listener = socket(AF_INET, SOCK_STREAM, 0);
    sv_addr.sin_family = AF_INET;
    // Mi metto in ascolto su tutte le interfacce (indirizzi IP)
    sv_addr.sin_addr.s_addr = INADDR_ANY;
    sv_addr.sin_port = htons(20000);
    bind(listener, (struct sockaddr*)& sv_addr, sizeof(sv_addr));
    listen(sd, 10);
```

I/O Multiplexing – Utilizzo di select() (1/2)

```
FD_SET(listener, &master); // Aggiungo il listener al set
fdmax = listener;          // Tengo traccia del maggiore
for(;;) {
    read_fds = master;      // Copia
    select(fdmax + 1, &read_fds, NULL, NULL, NULL);
    for(i = 0; i <= fdmax; i++) { // Scorro tutto il set
        if(FD_ISSET(i, &read_fds)) { // Trovato un desc. pronto
            if(i == listener) { // È il listener
                addrlen = sizeof(cl_addr);
                newfd = accept(listener, (struct sockaddr *)&cl_addr, &addrlen);
                FD_SET(newfd, &master); // Aggiungo il nuovo socket
                if(newfd > fdmax){ fdmax = newfd; } // Aggiorno max
            } else { // È un altro socket
                nbytes = recv(i, buf, sizeof(buf));
                //... Uso dati
                close(i); // Chiudo socket
                FD_CLR(i, &master); // Rimuovo il socket dal set
            }
        }
    }
}
return 0;
}
```

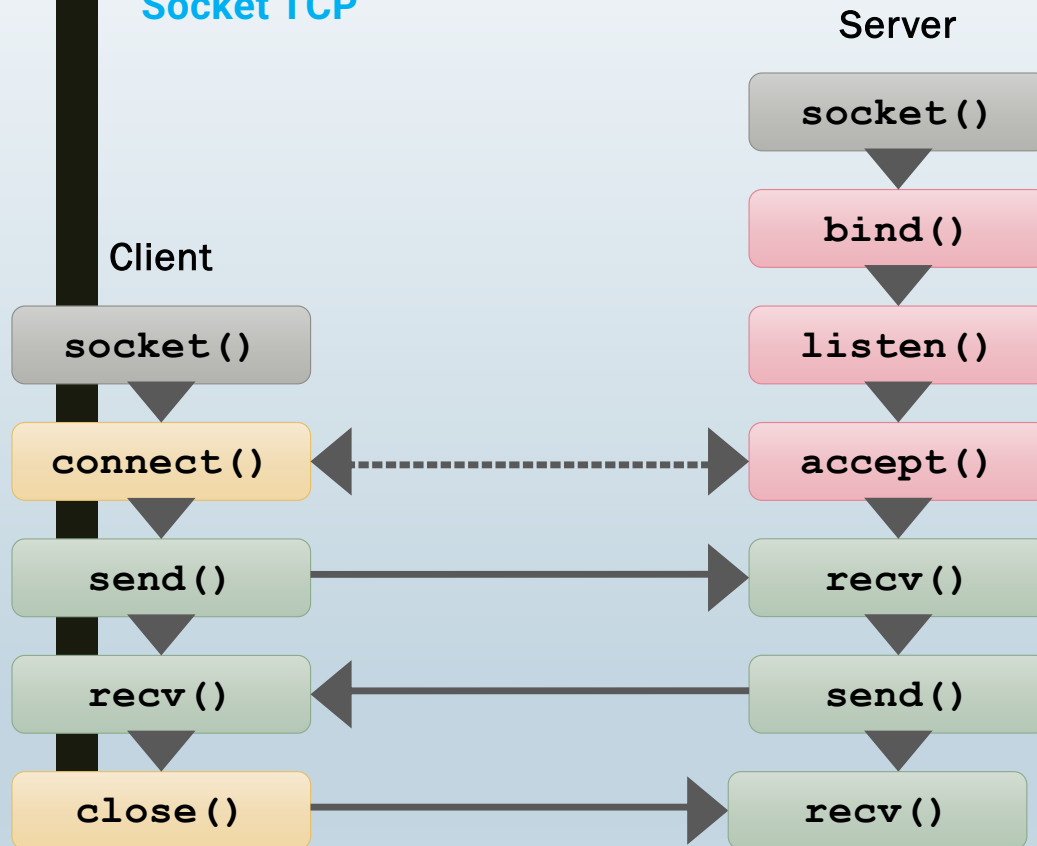
SOCKET UDP

TCP Vs UDP

- TCP instaura una connessione
 - Cioè prevede operazioni preliminari per instaurare un canale virtuale
 - Affidabile: i pacchetti inviati arrivano tutti, nell'ordine in cui sono stati inviati
 - Comporta latenza maggiore per il riordino e eventuali ritrasmissioni
- UDP è connection-less
 - Nessuna operazione preliminare
 - Rapido: nessun recupero e nessun riordino

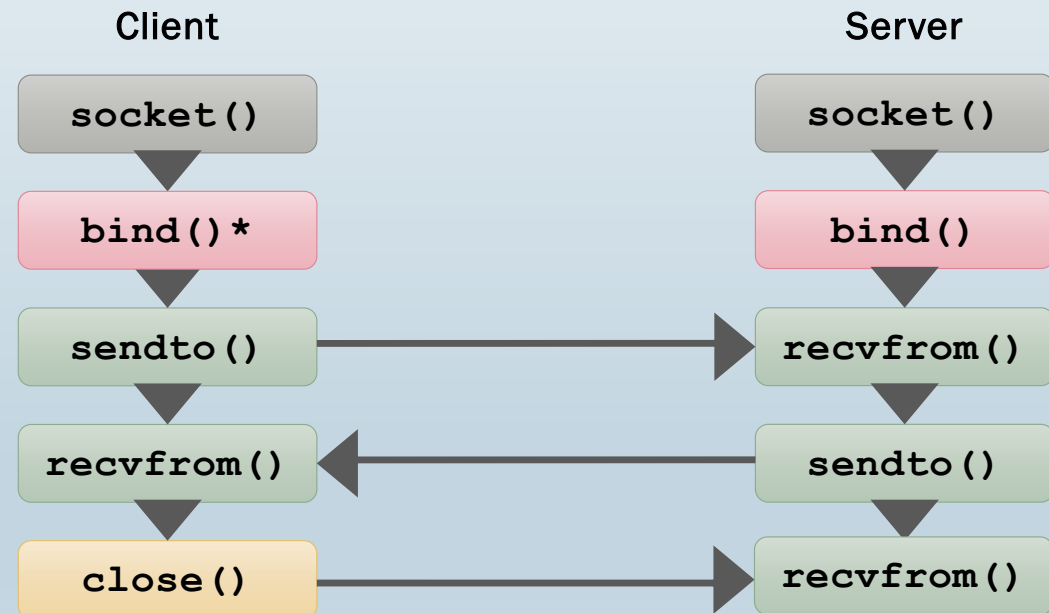
TCP Vs UDP

Socket TCP



Socket UDP

* La bind può anche essere opzionale. È necessaria se si vuole usare una porta specifica (per superare restrizioni di firewall o NAT)



Primitiva sendto()

- Invia un messaggio attraverso un socket all'indirizzo specificato

```
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, const struct sockaddr *dest_addr, socklen_t addrlen);
```

- **sockfd**: descrittore del socket
- **buf**: puntatore al buffer contenente il messaggio da inviare
- **len**: dimensione in byte del messaggio
- **flags**: per settare delle opzioni, lasciamolo a 0
- **dest_addr**: puntatore alla struttura in cui ho salvato l'indirizzo del destinatario
- **addrlen**: lunghezza di **dest_addr**
- La funzione restituisce il **numero di byte inviati**, -1 su errore
- **La funzione è bloccante**: il programma si ferma finché non ha scritto tutto il messaggio

Primitiva recvfrom()

- Riceve un messaggio attraverso un socket

```
ssize_t recvfrom(int sockfd, const void *buf, size_t len, int flags, struct sockaddr *src_addr, socklen_t addrlen);
```

- **sockfd**: descrittore del socket
- **buf**: puntatore al buffer contenente il messaggio da ricevere
- **len**: dimensione in byte del messaggio
- **flags**: per settare delle opzioni
- **src_addr**: puntatore a una struttura vuota in cui salvare l'indirizzo del mittente
- **addrlen**: lunghezza di src_addr
- La funzione restituisce il numero di byte ricevuti, -1 su errore, 0 se il socket remoto si è chiuso (vedi più avanti)
- **La funzione è bloccante**: il programma si ferma finché non ha letto qualcosa

Codice del server

```
int main () {
    int ret, sd, len;
    char buf[BUFLLEN];
    struct sockaddr_in my_addr, cl_addr;
    int addrlen = sizeof(cl_addr);
    /* Creazione socket */
    sd = socket(AF_INET, SOCK_DGRAM, 0);
    /* Creazione indirizzo */
    memset(&my_addr, 0, sizeof(my_addr)); // Pulizia
    my_addr.sin_family = AF_INET ;
    my_addr.sin_port = htons(4242);
    my_addr.sin_addr.s_addr = INADDR_ANY;
    ret = bind(sd, (struct sockaddr*)&my_addr, sizeof(my_addr));

    while(1) {
        len = recvfrom(sd, buf, BUFLLEN, 0, (struct sockaddr*)&cl_addr, &addrlen);
        //...
    }
}
```


Codice del client

```
int main () {
    int ret, sd, len;
    char buf[BUFLen];
    struct sockaddr_in my_addr, sv_addr; // Struttura per il server
    sd = socket(AF_INET, SOCK_DGRAM, 0); /* Creazione socket */
    if (sd < 0) {
        perror("Errore nella creazione del socket");
        exit(1);}
    memset(&my_addr, 0, sizeof(my_addr)); /* Creazione proprio indirizzo*/
    my_addr.sin_family = AF_INET;
    my_addr.sin_addr.s_addr = INADDR_ANY;
    my_addr.sin_port = htons(5432);
    if (bind(sd, (struct sockaddr*)&my_addr, sizeof(my_addr)) < 0) {
        perror("Errore nel bind");
        close(sd);
        exit(1);}
    memset(&sv_addr, 0, sizeof(sv_addr)); /* Creazione indirizzo del server */
    sv_addr.sin_family = AF_INET ;
    sv_addr.sin_port = htons(4242);
    inet_pton(AF_INET, "192.168.4.5", &sv_addr.sin_addr);
    while(1) {
        len = sendto(sd, buf, BUFLen, 0, (struct sockaddr*)&sv_addr, sizeof(sv_addr));
        //...
    }
}
```

Codice del client – senza bind

```
int main () {
    int ret, sd, len;
    char buf[BUFLen];
    struct sockaddr_in sv_addr; // Struttura per il server
    /* Creazione socket */
    sd = socket(AF_INET, SOCK_DGRAM, 0);
    /* Creazione indirizzo del server */
    memset(&sv_addr, 0, sizeof(sv_addr)); // Pulizia
    sv_addr.sin_family = AF_INET ;
    sv_addr.sin_port = htons(4242);
    inet_pton(AF_INET, "192.168.4.5", &sv_addr.sin_addr);

    while(1) {
        len = sendto(sd, buf, BUFLen, 0, (struct sockaddr*)&sv_addr, sizeof(sv_addr));
        //...
    }
}
```

Socket UDP «connesso»

- Usando **connect()** si può associare ad un socket UDP un indirizzo remoto
- Il socket riceverà/invierà pacchetti solo da/a quel indirizzo
 - **Non** è una connessione!
- Con un socket connesso si possono usare **send()** e **recv()**, evitando di specificare ogni volta l'indirizzo

